

TP1 - Grupo 12

Circuitos Digitales y Microcontroladores

Federico Goncalves

Joaquin Guzman

Microchip

main.c

En el main nos centramos en primero hacer todo el setup de todos los componentes, y luego en el loop pensamos que lo mejor sería que por cada iteración el programa evalúe por un lado si es necesario actualizar los leds o el neopixel, y por otro lado si alguno de los dos botones fue presionado.

Para el setup del main, configuramos los puertos como entrada o salida como correspondía, y las entradas de botones las seteamos con pull up interno.

Para mostrar correctamente la actualización del neopixel y de los leds, dado que uno debía actualizarse cada 100ms y otro cada 150ms vimos correcto que el código main tenga el loop con un delay de 50ms donde por cada iteración verifica si debe de actualizar algún componente. Para eso utilizamos dos variables llamadas **ld_iteration** y **np_iteration** las cuales se incrementan por cada iteración. El problema que tenía esta implementación era que daba muy poco margen para hacer polling con los botones. Por lo que pensamos que la mejor forma era reducir el delay y aumentar las iteraciones que debían hacerse cada componente.

```
#define LEDS_ITERATION 20 //100ms = leds_iteration * Refresh_rate_ms
#define NEO_ITERATION 30 // 150ms = Neo_iteration * Refresh_rate_ms
#define REFRESH_RATE_MS 5
```

```
ld_iteration++;
np_iteration++;
PORTD = ld_leds;
_delay_ms(REFRESH_RATE_MS);
}
```

De esta forma, los componentes se actualizan cada 100ms y 150ms, pero cada 5ms se verifica si se presiona algún botón.

Luego las variables de iteración se utilizan en conjunto con los **defines** de arriba para que efectivamente se actualicen los componentes cuando pasen las iteraciones definidas.

```

if (ld_Iteration>=LEDS_ITERATION){
    cambiarLeds();
    ld_Iteration=0;
}

if (np_Iteration>=NEO_ITERATION){
    neopixel();
    np_Iteration=0;
}

```

Botones

Para la implementación de los botones, se hizo una función llamada **botonPresionado** a la que se le envía el pin específico que debe utilizar, y retorna 1 o 0 si ve que el botón está presionado o no. Donde teniendo en cuenta que las entradas estaban con un pull up interno, sabemos que esta función evalúa lógicamente si la entrada está en HIGH o LOW, y retorna 1 si está en LOW o 0 si está en HIGH.

```

B1 = botonPresionado(PINC0);
if (B1_prev && !B1){
    ld_mod0= ld_mod0^1;

    if (ld_mod0){
        ld_leds = 0x80;
        ld_izq = 0;
    }else {
        ld_leds = 0x01;
        ld_izq = 1;
    }

    PORTD = ld_leds;
    ld_Iteration = 0;
}
B1_prev=B1;

```

Para evitar tener inconsistencias al mantener apretado el botón por un tiempo, utilizamos dos variables con el valor actual y valor anterior del botón para que solo detecte los flancos al cambiar de estado.

Leds

Para los leds se implementó la función **cambiarLeds** la cual modifica una variable de 8bits llamada **ld_leds** que representa el estado futuro de los leds, para no utilizar directamente el **PORTD** sino que se maneja la variable y luego se hace **PORTD = ld_leds**.

Variables

```
// Variables leds
static uint8_t ld_mod0 = 0;
static uint8_t ld_leds = 0x01;
static uint8_t ld_izq = 1;
static uint8_t ld_It0ration = 0;
```

Para el control de los leds se utilizaron 4 variables.

Una variable representa el modo o estado de los leds, que luego conmuta entre 0 y 1 para diferenciar los dos modos que deben tener los leds según el enunciado. La otra variable ya mencionada de los leds que comienza como especifica el enunciado con solo el led menos significativo encendido. Luego una variable llamada **ld_izq** que indica con 1 si los leds se desplazan hacia la izquierda y con 0 si se desplazan a la derecha. Y por último la variable **ld_It0ration** ya mencionada.

cambiarLeds()

El funcionamiento de **cambiarLeds** primero analiza los extremos de los leds según el estado en el que esté, y si no es un caso de los extremos se desplaza según la variable **ld_izq**. Para el extremo derecho siempre se va a desplazar hacia la izquierda, pero para el extremo izquierdo se tiene en cuenta el modo en el que está, para así pasar al extremo derecho en caso de estar en modo desplazamiento, o pasar al led anterior si está en modo de rebote. Destacar que esta función nunca toca ningún componente, solamente actúa sobre una variable que luego fuera de la función se utiliza en el **PORTD**.

```
void cambiarLeds(){
    if (ld_leds == 0x01){ //Extremo derecho
        ld_izq = 1;
        ld_leds <<=1;
        return;
    }
    if (ld_leds == 0x80){ //Extremo izquierdo
        if (ld_mod0){
            ld_izq = 0;
            ld_leds >>= 1;
        }else {
            ld_leds = 0x01;
        }
        return;
    }
    if (ld_izq) ld_leds <<= 1; // Desplazar a Izquierda o Derecha
    else ld_leds >>= 1;
    return;
}
```

Neopixel

Para implementar la lógica de funcionamiento pedida se implementó la función **neopixel**. Esta se encarga de enviar al neopixel la secuencia de bytes correspondiente a cada led de cada uno de los 8 píxeles.

```
#define NOP() __asm__ __volatile__("nop\n\t")
```

Se definió **NOP** como función que agrega un nop al código de máquina sin que el compilador lo optimice.

Variables

```
// Variables Neopixel
static uint8_t np_mod0 = 0;
static uint8_t np_pos = 0;
static uint8_t np_paridad = 0;
static uint8_t np_Iteration = 0;
```

Para el control del neopixel se utilizaron 4 variables:

- np_mod0: Variable de 8bits, el modo de funcionamiento correspondiente al inciso c se indica como np_mod0 = 0 mientras que el correspondiente al inciso d es np_mod0=255.
- np_pos: Variable auxiliar para **modoD** la cuál indica la posición del led a prender en esa iteración de **neopixel**.
- np_paridad: Variable auxiliar para **modoC** la cuál indica si son los leds pares o impares los que han de prenderse en esa iteración de **neopixel**.
- np_Iteration: Variable que permite contar la cantidad de loops de **REFRESH_RATE_MS** segundos que se realizaron.

neopixel()

```
void neopixel(){
    // Modo C
    if(np_modo == 0){
        for (int i=0; i<4; i++){
            enviar_byte(0);
            enviar_byte(0);
            enviar_byte((255 & (np_paridad)));

            enviar_byte(0);
            enviar_byte((255 & ~(np_paridad)));
            enviar_byte(0);
        }
        np_paridad = ~np_paridad;
    }
    // Modo D
    else {
        for (int i = 0; i < 8; i++){
            enviar_byte(255 * (i == 8 - np_pos));
            enviar_byte(0);
            enviar_byte(0);
        }
        np_pos++;
        if (np_pos > 8) np_pos = 0;
    }
}
```

En caso de que **np_modo** sea 0 esto significa que el modo de funcionamiento del neopixel es **modoC** y que por lo tanto han de prenderse de manera alternada los leds rojos pares y los leds azules impares. El trackeo de paridad se realiza con la variable **np_paridad**.

Caso contrario, el modo de funcionamiento se encuentra configurado en **modoD** y por lo tanto habrán de prenderse de derecha a izquierda los leds verdes. El trackeo de la posición del led que 'toca' se hace con la variable **np_pos**.

enviar_byte()

```
void enviar_byte(uint8_t n){
    for (uint8_t i=0; i<8; i++){ // Cada iteración: Suma 5 Ciclos ->
        // Última iteración: Suma 4 Ciclos

        // BIT 1
        if(n & 0x80){ // Suma ~3 Ciclos (Si bit 1 -> 2 Ciclos, si bit 0 -> 3 Ciclos

            // T1H = 12 Ciclos -> 750ns
            // T1L = 9 Ciclos -> 562,5ns

            PORTB |= (1 << PORTB0); // Suma 3 Ciclos -> 187,5ns
            NOP();NOP();NOP();NOP();NOP();NOP();NOP();NOP(); // Suma 9 Ciclos -> 562,5ns
            PORTB &= ~(1 << PORTB0); // Suma 3 Ciclos -> 187,5ns

        }

        // BIT 0
        else {

            // T0H = 6 Ciclos -> 375ns
            // T0L = 14 Ciclos -> 875ns

            PORTB |= (1 << PORTB0); // Suma 3 Ciclos -> 187,5ns
            NOP();NOP();NOP(); // Suma 3 Ciclos -> 187,5ns
            PORTB &= ~(1 << PORTB0); // Suma 3 Ciclos -> 187,5ns
            NOP();NOP();NOP();NOP();NOP(); // Suma 5 Ciclos -> 312,5ns

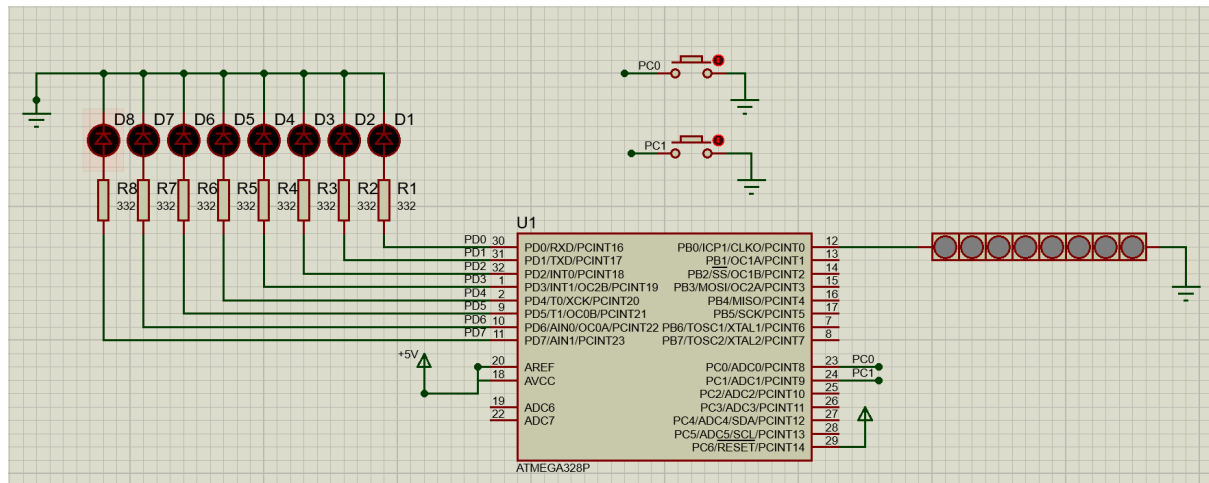
        }

        n <<=1; // Suma 1 Ciclo -> 62,5ns
    }
}
```

Esta función se encarga del envío de bytes al neopixel teniendo en cuenta la codificación de bit 1 y 0 especificada en la datasheet. Para esta implementación se tuvo en cuenta el tiempo de ejecución de instrucciones auxiliares que se agrega al tiempo de HIGH y LOW del puerto. Se indica entonces en comentarios cuántos ciclos de CPU agrega cada instrucción por iteración (esto se realizó analizando el código de máquina generado) y se tienen en cuenta para la comunicación con el neopixel.

Proteus

Esquema general:



Para el conexionado del Neopixel, según la datasheet proporcionada utiliza un puerto como entrada de datos, y se alimenta del mismo. La otra terminal se utiliza para conectar a GND o para encadenar otras tiras de Neopixel en serie.

Para los leds utilizamos 8 leds rojos configurados en Proteus como Analógicos y para calcular las resistencias necesarias para los diodos planteamos que la corriente máxima que pasen por estas sea de 10mA, y el voltaje para su funcionamiento de 1.8V. Si nosotros tenemos 5V en cada pin, necesitamos una caída de voltaje de $5V - 1.8V = 3.2V$. Con esa caída de voltaje y 10mA de corriente máxima obtenemos entonces que la resistencia necesaria es de $3.2V / 10mA = 320\Omega$. No encontramos resistencias exactas de 320Ω por lo que utilizamos resistencias de 332Ω que eran lo más cercano y nos aseguraba no llegar a 10mA de corriente máxima.